analysis

May 31, 2026

1 London Smart Meter — Energy & Weather Analysis

Household: MAC000002
Energy data: Half-hourly kWh readings, Oct 2012 – Feb 2014
Weather (hourly): Temperature, wind speed, humidity, visibility — Heathrow area
Weather (daily): Sunshine hours, precipitation, cloud cover, snow depth — Heathrow (Kaggle: emmanuelfwerr/london-weather-data)

1.0.1 Objectives

1. Identify dependencies between weather parameters and electricity consumption
2. Engineer time-series features (lags, rolling averages, time-of-day indicators)
3. Test for stationarity and decompose the consumption series
4. Build AR, MA, and ARMA forecasting models with 48-hour horizon

---

1.1 Section 1 — Data Loading & Cleaning

[1]:
```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.ar_model import AutoReg
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tools.sm_exceptions import ConvergenceWarning
from sklearn.metrics import mean_absolute_error, mean_squared_error

sns.set_theme(style='whitegrid', palette='muted', font_scale=1.1)
plt.rcParams['figure.dpi'] = 110
```

```
print('Libraries loaded.')
```

```
Libraries loaded.
```

1.1.1 1.1 Load Energy Data (half-hourly)

[2]:
```
energy_raw = pd.read_csv(
    'MAC000002_energy_halfhourly.csv',
    parse_dates=['time']
)

# Strip whitespace and coerce to float
energy_raw['energy(kWh/hh)'] = pd.to_numeric(
    energy_raw['energy(kWh/hh)'].astype(str).str.strip(), errors='coerce'
)
energy_raw = energy_raw.sort_values('time').reset_index(drop=True)

print(f'Shape: {energy_raw.shape}')
print(f'Date range: {energy_raw["time"].min()} → {energy_raw["time"].max()}')
print(f'Missing values:\n{energy_raw.isnull().sum()}')
energy_raw.head()
```

```
Shape: (24141, 3)
Date range: 2012-10-12 00:30:00 → 2014-02-28 00:00:00
Missing values:
time              0
LCLid             0
energy(kWh/hh)    1
dtype: int64
```

[2]:

	time	LCLid	energy(kWh/hh)
0	2012-10-12 00:30:00	MAC000002	0.0
1	2012-10-12 01:00:00	MAC000002	0.0
2	2012-10-12 01:30:00	MAC000002	0.0
3	2012-10-12 02:00:00	MAC000002	0.0
4	2012-10-12 02:30:00	MAC000002	0.0

[3]:
```
# Drop the single null row
energy_raw = energy_raw.dropna(subset=['energy(kWh/hh)'])

# Align half-hour readings to complete hourly bins
# (00:30 and 01:00 belong to the 00:00-01:00 hour, etc.)
energy_hourly = (
    energy_raw
    .assign(time_hour=energy_raw['time'] - pd.Timedelta(minutes=30))
    .set_index('time_hour')['energy(kWh/hh)']
    .resample('h')
    .sum()
```

```
    .rename('energy_kwh')
    .reset_index()
    .rename(columns={'time_hour': 'time'})
)

print(f'Hourly energy shape: {energy_hourly.shape}')
energy_hourly.head()
```

```
Hourly energy shape: (12096, 2)
```

[3]:

	time	energy_kwh
0	2012-10-12 00:00:00	0.0
1	2012-10-12 01:00:00	0.0
2	2012-10-12 02:00:00	0.0
3	2012-10-12 03:00:00	0.0
4	2012-10-12 04:00:00	0.0

1.1.2 1.2 Load Hourly Weather Data

[4]:
```
weather_hourly = pd.read_csv(
    'London_weather_hourly.csv',
    parse_dates=['time']
)

# Keep only the columns we need from hourly weather
hourly_cols = ['time', 'temperature', 'windSpeed', 'humidity', 'pressure']
weather_hourly = weather_hourly[hourly_cols].copy()

# Fill the 13 missing pressure values with forward-fill
weather_hourly['pressure'] = weather_hourly['pressure'].ffill()

print(f'Shape: {weather_hourly.shape}')
print(f'Date range: {weather_hourly["time"].min()} → {weather_hourly["time"].
 ↪max()}')
print(f'Missing values:\n{weather_hourly.isnull().sum()}')
weather_hourly.head()
```

```
Shape: (21165, 5)
Date range: 2011-11-01 00:00:00 → 2014-03-31 22:00:00
Missing values:
time           0
temperature    0
windSpeed      0
humidity       0
pressure       0
dtype: int64
```

[4]:
```
                  time  temperature  windSpeed  humidity  pressure
0 2011-11-11 00:00:00        10.24       2.77      0.91   1016.76
1 2011-11-11 01:00:00         9.76       2.95      0.94   1016.63
2 2011-11-11 02:00:00         9.46       3.17      0.96   1016.36
3 2011-11-11 03:00:00         9.23       3.25      0.96   1016.28
4 2011-11-11 04:00:00         9.26       3.70      1.00   1015.98
```

1.1.3 1.3 Load Daily Weather Data

This dataset provides **sunshine hours**, **precipitation (mm)**, **cloud cover (oktas)**, and **snow depth (cm)** — columns not available at hourly resolution. These are merged as daily-level supplements; each daily value is broadcast to every hour of that day.

[5]:
```python
weather_daily = pd.read_csv('london_weather.csv')

# Parse integer YYYYMMDD date column
weather_daily['date'] = pd.to_datetime(
    weather_daily['date'].astype(str), format='%Y%m%d'
)

# Keep only relevant daily columns
daily_cols = ['date', 'sunshine', 'precipitation', 'cloud_cover', 'snow_depth']
weather_daily = weather_daily[daily_cols].copy()

# Fill snow_depth nulls with 0 (no snow recorded)
weather_daily['snow_depth'] = weather_daily['snow_depth'].fillna(0)
# Fill remaining nulls with forward-fill
weather_daily = weather_daily.ffill()

print(f'Shape: {weather_daily.shape}')
print(f'Date range: {weather_daily["date"].min()} → {weather_daily["date"].
 ↪max()}')
print(f'Missing values:\n{weather_daily.isnull().sum()}')
weather_daily.head()
```

```
Shape: (15341, 5)
Date range: 1979-01-01 00:00:00 → 2020-12-31 00:00:00
Missing values:
date             0
sunshine         0
precipitation    0
cloud_cover      0
snow_depth       0
dtype: int64
```

[5]:
```
        date  sunshine  precipitation  cloud_cover  snow_depth
0 1979-01-01       7.0            0.4          2.0         9.0
1 1979-01-02       1.7            0.0          6.0         8.0
```

```
2 1979-01-03       0.0              0.0          5.0          4.0
3 1979-01-04       0.0              0.0          8.0          2.0
4 1979-01-05       2.0              0.0          6.0          1.0
```

1.1.4 1.4 Merge All Three Datasets

Strategy: 1. Left-join energy (hourly) ← hourly weather on `time` 2. Add a `date` key to the result, then left-join ← daily weather on `date`

All rows from the energy dataset are preserved.

[6]:
```python
# Step 1: energy ← hourly weather (left join)
df = energy_hourly.merge(weather_hourly, on='time', how='left')

# Step 2: add date key and left-join daily weather
df['date'] = df['time'].dt.normalize()  # floor to midnight
df = df.merge(weather_daily, on='date', how='left')

# Drop the helper date column (keep 'time' as the primary index)
df = df.drop(columns=['date'])
df = df.set_index('time').sort_index()

print(f'Final merged shape: {df.shape}')
print(f'Date range: {df.index.min()} → {df.index.max()}')
print(f'\nMissing values per column:')
print(df.isnull().sum())
df.head()
```

```
Final merged shape: (12096, 9)
Date range: 2012-10-12 00:00:00 → 2014-02-27 23:00:00

Missing values per column:
energy_kwh       0
temperature      2
windSpeed        2
humidity         2
pressure         2
sunshine         0
precipitation    0
cloud_cover      0
snow_depth       0
dtype: int64
```

[6]:
```
                     energy_kwh  temperature  windSpeed  humidity  pressure  \
time
2012-10-12 00:00:00         0.0        13.61       5.40      0.91    999.47
2012-10-12 01:00:00         0.0        13.09       6.77      0.89   1000.10
2012-10-12 02:00:00         0.0        12.54       6.46      0.86   1000.51
2012-10-12 03:00:00         0.0        11.94       6.37      0.84   1000.73
```

```
2012-10-12 04:00:00         0.0        11.47       6.63       0.86   1001.23

                     sunshine  precipitation  cloud_cover  snow_depth
time
2012-10-12 00:00:00       6.4            0.0          5.0         0.0
2012-10-12 01:00:00       6.4            0.0          5.0         0.0
2012-10-12 02:00:00       6.4            0.0          5.0         0.0
2012-10-12 03:00:00       6.4            0.0          5.0         0.0
2012-10-12 04:00:00       6.4            0.0          5.0         0.0
```

[7]:

```python
# Any remaining NaNs (hours before weather coverage) - drop them
before = len(df)
df = df.dropna()
after = len(df)
print(f'Dropped {before - after} rows with missing values (outside weather␣
 ↪coverage).')
print(f'Final analysis DataFrame: {df.shape}')
print(f'Date range: {df.index.min()} → {df.index.max()}')
df.describe().round(3)
```

```
Dropped 2 rows with missing values (outside weather coverage).
Final analysis DataFrame: (12094, 9)
Date range: 2012-10-12 00:00:00 → 2014-02-27 23:00:00
```

[7]:

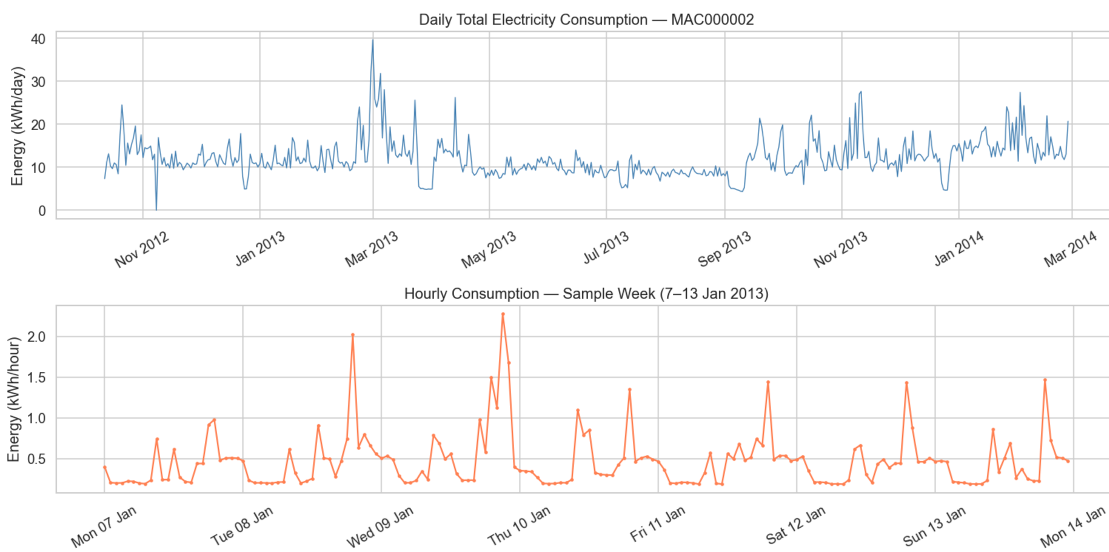
	energy_kwh	temperature	windSpeed	humidity	pressure	sunshine
count	12094.000	12094.000	12094.000	12094.000	12094.000	12094.000
mean	0.504	9.898	4.002	0.787	1012.489	3.566
std	0.448	5.933	2.103	0.138	11.416	3.716
min	0.000	-3.860	0.040	0.230	975.740	0.000
25%	0.218	5.660	2.450	0.710	1005.460	0.200
50%	0.341	9.145	3.770	0.820	1013.465	2.500
75%	0.606	13.650	5.200	0.890	1020.550	5.675
max	4.780	32.400	14.800	1.000	1040.130	14.500

	precipitation	cloud_cover	snow_depth
count	12094.000	12094.000	12094.000
mean	2.028	4.717	0.026
std	3.876	2.311	0.304
min	0.000	0.000	0.000
25%	0.000	3.000	0.000
50%	0.200	5.000	0.000
75%	2.400	7.000	0.000
max	29.800	8.000	5.000

1.2 Section 2 — Exploratory Data Analysis (EDA)

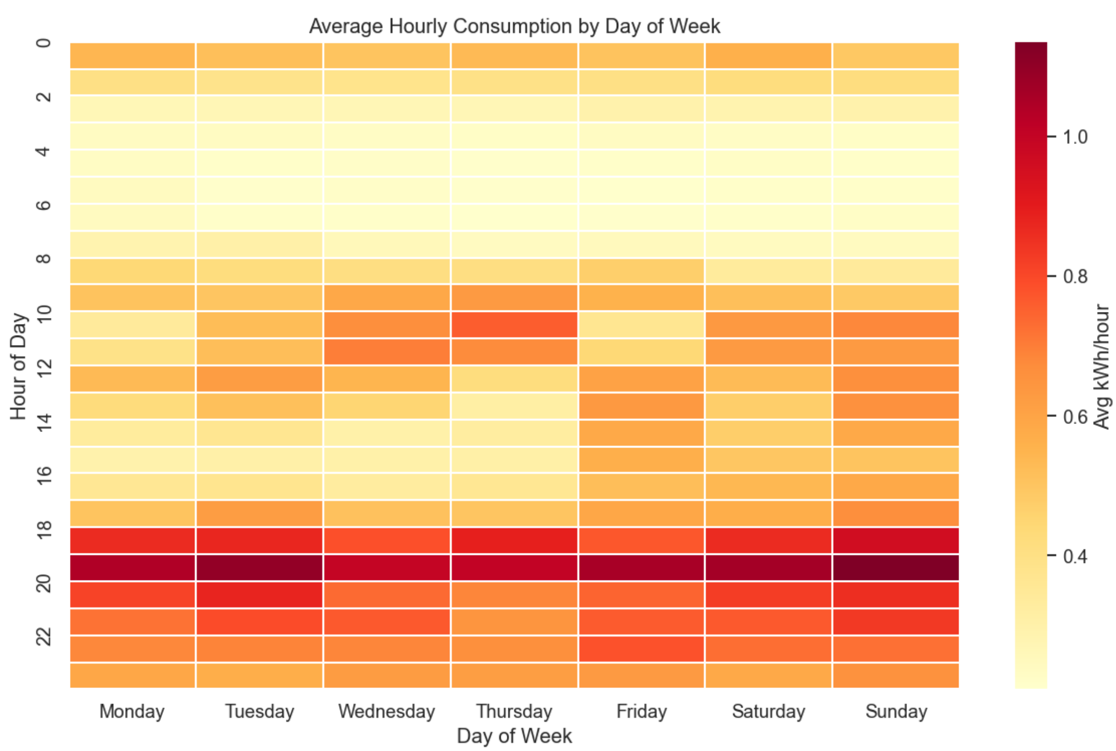
1.2.1 2.1 Consumption Over Time

[8]:
```
daily_energy = df['energy_kwh'].resample('D').sum()

fig, axes = plt.subplots(2, 1, figsize=(14, 7), sharex=False)

# Full period - daily total
axes[0].plot(daily_energy.index, daily_energy.values, color='steelblue',␣
↪linewidth=0.9)
axes[0].set_title('Daily Total Electricity Consumption - MAC000002')
axes[0].set_ylabel('Energy (kWh/day)')
axes[0].xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))

# One representative week (first full week of Jan 2013)
week = df['energy_kwh']['2013-01-07':'2013-01-13']
axes[1].plot(week.index, week.values, color='coral', linewidth=1.4, marker='o',␣
↪markersize=2)
axes[1].set_title('Hourly Consumption - Sample Week (7-13 Jan 2013)')
axes[1].set_ylabel('Energy (kWh/hour)')
axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%a %d %b'))

for ax in axes:
    ax.tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```



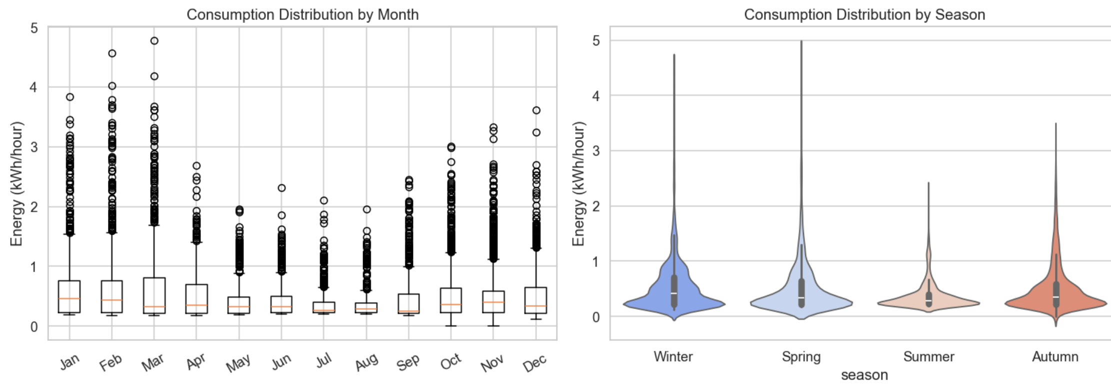
1.2.2 2.2 Heatmap — Consumption by Hour of Day × Day of Week

[9]:
```
df_tmp = df.copy()
df_tmp['hour'] = df_tmp.index.hour
df_tmp['dayofweek'] = df_tmp.index.day_name()

day_order =␣
  ↪['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
pivot = df_tmp.pivot_table(
    values='energy_kwh', index='hour', columns='dayofweek', aggfunc='mean'
)[day_order]

fig, ax = plt.subplots(figsize=(11, 7))
sns.heatmap(
    pivot, cmap='YlOrRd', ax=ax, linewidths=0.3,
    cbar_kws={'label': 'Avg kWh/hour'}
)
ax.set_title('Average Hourly Consumption by Day of Week')
ax.set_xlabel('Day of Week')
ax.set_ylabel('Hour of Day')
ax.set_yticks(range(0, 24, 2))
ax.set_yticklabels(range(0, 24, 2))
plt.tight_layout()
plt.show()
```



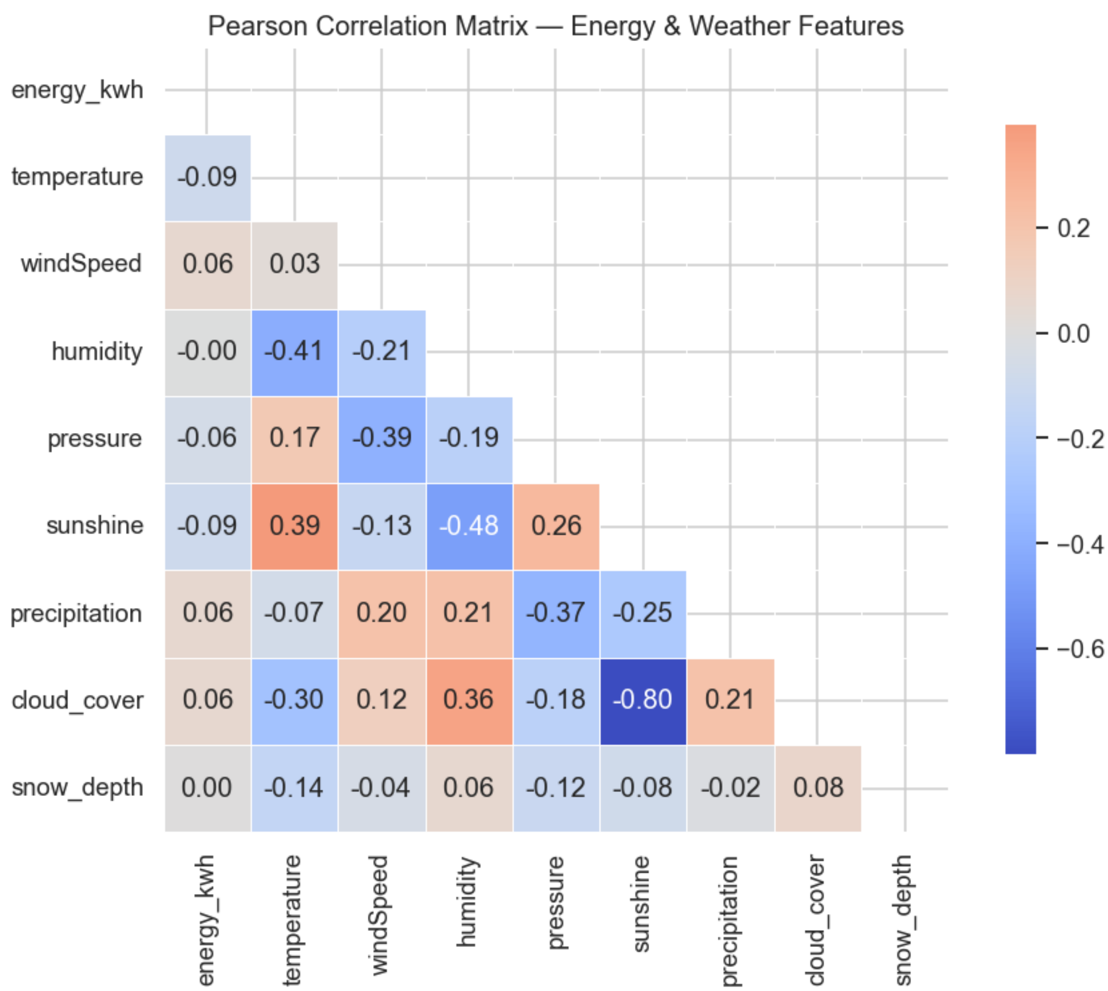
1.2.3 2.3 Seasonal Consumption Patterns

[10]:
```
df_tmp = df.copy()
df_tmp['month'] = df_tmp.index.month
df_tmp['season'] = df_tmp['month'].map({
    12:'Winter', 1:'Winter', 2:'Winter',
    3:'Spring', 4:'Spring', 5:'Spring',
    6:'Summer', 7:'Summer', 8:'Summer',
    9:'Autumn', 10:'Autumn', 11:'Autumn'
})

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Box plot by month
monthly = df_tmp.groupby('month')['energy_kwh'].apply(list)
month_labels =␣
↪['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
axes[0].boxplot(
    [df_tmp[df_tmp['month']==m]['energy_kwh'].values for m in range(1,13)],
    labels=month_labels
)
axes[0].set_title('Consumption Distribution by Month')
axes[0].set_ylabel('Energy (kWh/hour)')
axes[0].tick_params(axis='x', rotation=30)

# Violin by season
season_order = ['Winter','Spring','Summer','Autumn']
sns.violinplot(
    data=df_tmp, x='season', y='energy_kwh',
    order=season_order, palette='coolwarm', ax=axes[1]
)
axes[1].set_title('Consumption Distribution by Season')
axes[1].set_ylabel('Energy (kWh/hour)')

plt.tight_layout()
plt.show()
```



1.2.4 2.4 Correlation Heatmap — Weather vs. Consumption

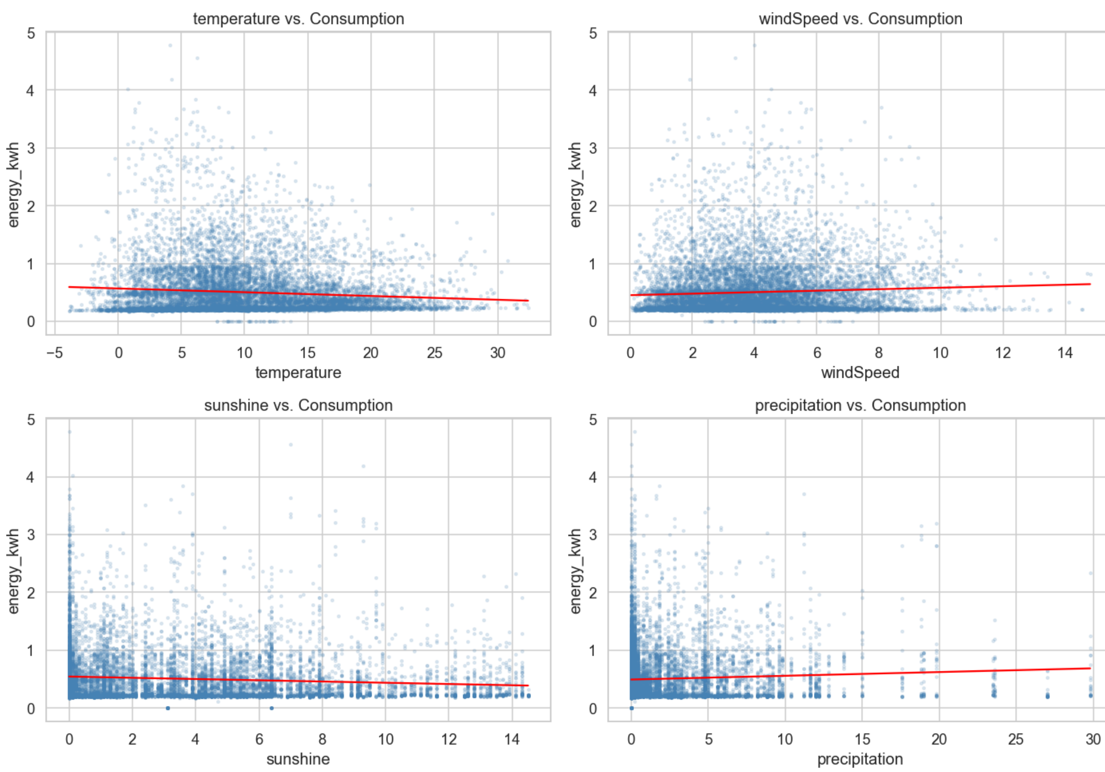
[11]:
```python
corr_cols = ['energy_kwh', 'temperature', 'windSpeed', 'humidity',
             'pressure', 'sunshine', 'precipitation', 'cloud_cover',␣
 ↪'snow_depth']
corr = df[corr_cols].corr()

fig, ax = plt.subplots(figsize=(9, 7))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(
    corr, mask=mask, annot=True, fmt='.2f',
    cmap='coolwarm', center=0, square=True,
    linewidths=0.5, ax=ax, cbar_kws={'shrink': 0.8}
)
ax.set_title('Pearson Correlation Matrix - Energy & Weather Features')
plt.tight_layout()
plt.show()

print('\nCorrelation with energy_kwh (sorted):')
print(corr['energy_kwh'].drop('energy_kwh').sort_values())
```




```
Correlation with energy_kwh (sorted):
sunshine        -0.089239
temperature     -0.086442
pressure        -0.059693
humidity        -0.003726
snow_depth       0.004886
cloud_cover      0.055478
precipitation    0.055961
windSpeed        0.060502
Name: energy_kwh, dtype: float64
```

1.2.5 2.5 Scatter Plots — Key Weather Variables vs. Consumption

[12]:
```python
focus = ['temperature', 'windSpeed', 'sunshine', 'precipitation']
fig, axes = plt.subplots(2, 2, figsize=(13, 9))

for ax, col in zip(axes.flatten(), focus):
    ax.scatter(df[col], df['energy_kwh'], alpha=0.15, s=5, color='steelblue')
    # Trend line
    z = np.polyfit(df[col].dropna(), df.loc[df[col].notna(), 'energy_kwh'], 1)
    p = np.poly1d(z)
    xs = np.linspace(df[col].min(), df[col].max(), 200)
    ax.plot(xs, p(xs), color='red', linewidth=1.5)
    ax.set_xlabel(col)
    ax.set_ylabel('energy_kwh')
    ax.set_title(f'{col} vs. Consumption')

plt.tight_layout()
plt.show()
```



1.3 Section 3 — Feature Engineering

[13]:
```python
df_feat = df.copy()

# --- Time features ---
df_feat['hour']       = df_feat.index.hour
df_feat['dayofweek']  = df_feat.index.dayofweek   # 0=Monday
df_feat['month']      = df_feat.index.month
df_feat['is_weekend'] = (df_feat['dayofweek'] >= 5).astype(int)
df_feat['season']     = df_feat['month'].map({
    12:0, 1:0, 2:0, 3:1, 4:1, 5:1,
    6:2, 7:2, 8:2, 9:3, 10:3, 11:3
})  # 0=Winter, 1=Spring, 2=Summer, 3=Autumn

# --- Lagged temperature (1h, 24h, 48h) ---
df_feat['temp_lag1h']  = df_feat['temperature'].shift(1)
df_feat['temp_lag24h'] = df_feat['temperature'].shift(24)
df_feat['temp_lag48h'] = df_feat['temperature'].shift(48)

# --- Rolling (moving) averages of consumption ---
df_feat['energy_ma24h'] = df_feat['energy_kwh'].rolling(window=24,␣
↪min_periods=1).mean()
df_feat['energy_ma48h'] = df_feat['energy_kwh'].rolling(window=48,␣
↪min_periods=1).mean()
df_feat['energy_ma168h'] = df_feat['energy_kwh'].rolling(window=168,␣
↪min_periods=1).mean()  # 1 week

# --- Lagged consumption (previous day, previous week) ---
df_feat['energy_lag24h']  = df_feat['energy_kwh'].shift(24)
df_feat['energy_lag168h'] = df_feat['energy_kwh'].shift(168)

print(f'Feature-engineered DataFrame: {df_feat.shape}')
print(f'Columns: {list(df_feat.columns)}')
df_feat.head(3)
```

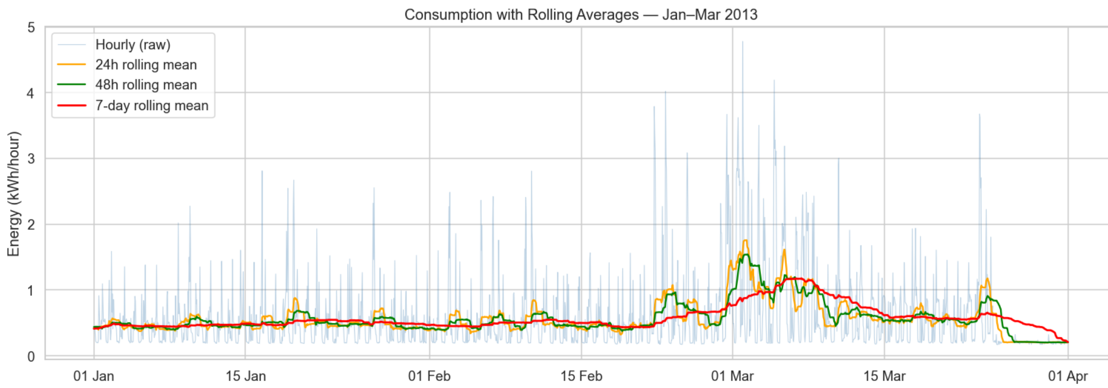
```
Feature-engineered DataFrame: (12094, 22)
Columns: ['energy_kwh', 'temperature', 'windSpeed', 'humidity', 'pressure',
'sunshine', 'precipitation', 'cloud_cover', 'snow_depth', 'hour', 'dayofweek',
'month', 'is_weekend', 'season', 'temp_lag1h', 'temp_lag24h', 'temp_lag48h',
'energy_ma24h', 'energy_ma48h', 'energy_ma168h', 'energy_lag24h',
'energy_lag168h']
```

[13]:
```
                     energy_kwh  temperature  windSpeed  humidity  pressure  \
time
2012-10-12 00:00:00         0.0        13.61       5.40      0.91    999.47
2012-10-12 01:00:00         0.0        13.09       6.77      0.89   1000.10
2012-10-12 02:00:00         0.0        12.54       6.46      0.86   1000.51
```

```
                     sunshine  precipitation  cloud_cover  snow_depth  hour  \
time
2012-10-12 00:00:00       6.4            0.0          5.0         0.0     0
2012-10-12 01:00:00       6.4            0.0          5.0         0.0     1
2012-10-12 02:00:00       6.4            0.0          5.0         0.0     2

                     …  is_weekend  season  temp_lag1h  temp_lag24h  \
time                 …
2012-10-12 00:00:00  …           0       3         NaN          NaN
2012-10-12 01:00:00  …           0       3       13.61          NaN
2012-10-12 02:00:00  …           0       3       13.09          NaN

                     temp_lag48h  energy_ma24h  energy_ma48h  energy_ma168h  \
time
2012-10-12 00:00:00          NaN           0.0           0.0            0.0
2012-10-12 01:00:00          NaN           0.0           0.0            0.0
2012-10-12 02:00:00          NaN           0.0           0.0            0.0

                     energy_lag24h  energy_lag168h
time
2012-10-12 00:00:00            NaN             NaN
2012-10-12 01:00:00            NaN             NaN
2012-10-12 02:00:00            NaN             NaN

[3 rows x 22 columns]
```

1.3.1 3.1 Rolling Averages — Visual Check

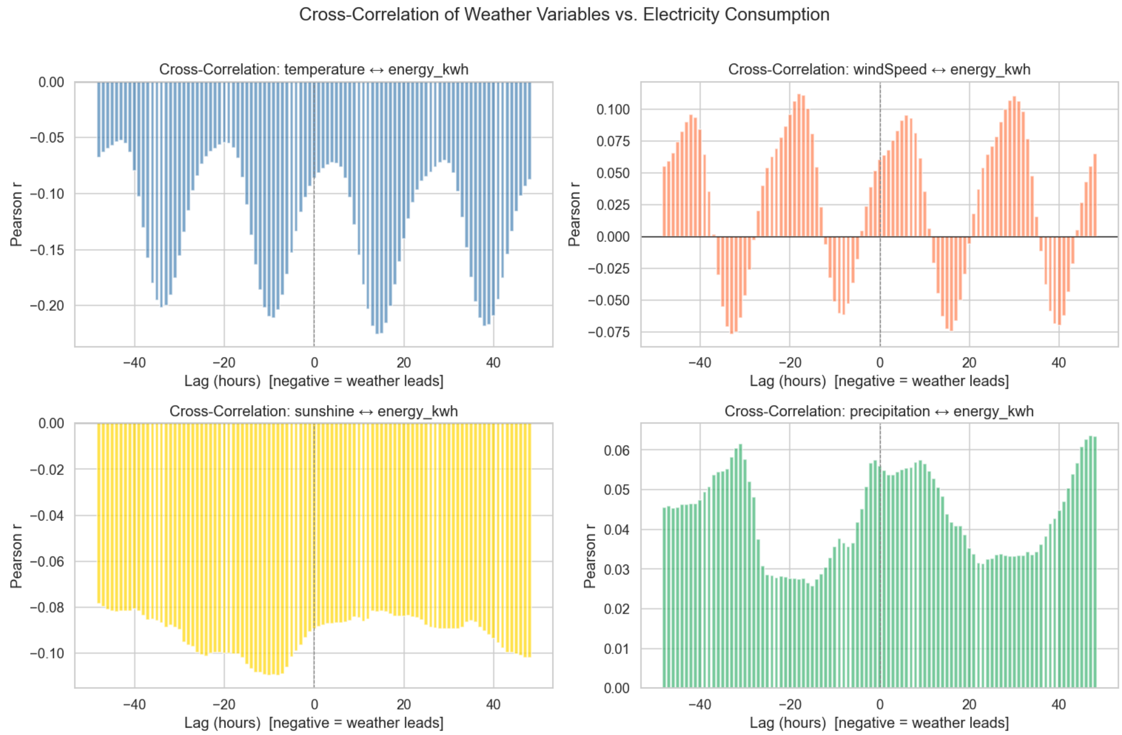
[14]:
```
sample = df_feat['2013-01-01':'2013-03-31']

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(sample.index, sample['energy_kwh'],   alpha=0.35, linewidth=0.7,␣
 ↪label='Hourly (raw)', color='steelblue')
ax.plot(sample.index, sample['energy_ma24h'],  linewidth=1.5, label='24h␣
 ↪rolling mean', color='orange')
ax.plot(sample.index, sample['energy_ma48h'],  linewidth=1.5, label='48h␣
 ↪rolling mean', color='green')
ax.plot(sample.index, sample['energy_ma168h'], linewidth=1.8, label='7-day␣
 ↪rolling mean', color='red')
ax.set_title('Consumption with Rolling Averages - Jan-Mar 2013')
ax.set_ylabel('Energy (kWh/hour)')
ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d %b'))
plt.tight_layout()
plt.show()
```



1.3.2 3.2 Cross-Correlation Analysis — Weather Variables vs. Consumption at Multiple Lags

[15]:
```python
def cross_correlation(x, y, max_lag=48):
    """Return Pearson correlation between y and x shifted by each lag."""
    lags = range(-max_lag, max_lag + 1)
    corrs = []
    for lag in lags:
        if lag == 0:
            corrs.append(x.corr(y))
        elif lag > 0:
            corrs.append(x.shift(lag).corr(y))
        else:
            corrs.append(x.corr(y.shift(-lag)))
    return list(lags), corrs

weather_vars = ['temperature', 'windSpeed', 'sunshine', 'precipitation']
colors = ['steelblue', 'coral', 'gold', 'mediumseagreen']

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
for ax, var, col in zip(axes.flatten(), weather_vars, colors):
    lags, corrs = cross_correlation(df_feat[var].dropna(),␣
 ↪df_feat['energy_kwh'].dropna())
    ax.bar(lags, corrs, width=0.8, color=col, alpha=0.7)
    ax.axhline(0, color='black', linewidth=0.8)
    ax.axvline(0, color='grey', linewidth=0.8, linestyle='--')
    ax.set_title(f'Cross-Correlation: {var}   energy_kwh')
    ax.set_xlabel('Lag (hours)  [negative = weather leads]')
    ax.set_ylabel('Pearson r')

plt.suptitle('Cross-Correlation of Weather Variables vs. Electricity␣
 ↪Consumption', y=1.01)
plt.tight_layout()
```

```
plt.show()
```



1.4 Section 4 — Time Series Analysis

We analyse the **hourly consumption series** for stationarity, then decompose it into trend, seasonal, and residual components. ACF and PACF plots are used to guide AR(p) and MA(q) order selection for the forecasting models.

1.4.1 4.1 Stationarity — Augmented Dickey-Fuller Test

[16]:
```
series = df['energy_kwh'].copy()

def adf_report(s, label):
    result = adfuller(s.dropna(), autolag='AIC')
    print(f'--- ADF Test: {label} ---')
    print(f'  Test statistic : {result[0]:.4f}')
    print(f'  p-value        : {result[1]:.6f}')
    print(f'  Lags used      : {result[2]}')
    print(f'  Critical values: { {k: f"{v:.3f}" for k,v in result[4].items()}␣
↪}')
    if result[1] < 0.05:
        print('    STATIONARY at 5% significance level')
```

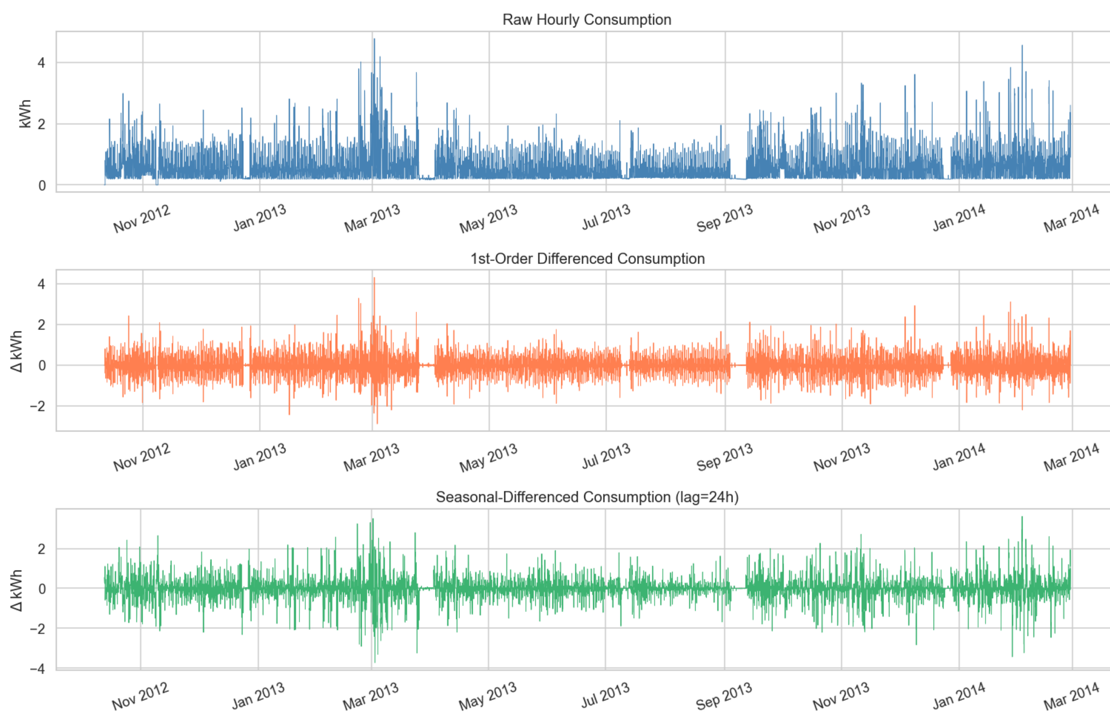
```
    else:
        print('    NON-STATIONARY - differencing required')
    print()

adf_report(series, 'Raw hourly consumption')

# First-order differencing
series_diff1 = series.diff().dropna()
adf_report(series_diff1, '1st-order differenced consumption')

# Seasonal differencing (lag=24)
series_sdiff = series.diff(24).dropna()
adf_report(series_sdiff, 'Seasonal-differenced consumption (lag=24h)')
```

```
--- ADF Test: Raw hourly consumption ---
  Test statistic : -11.4013
  p-value        : 0.000000
  Lags used      : 40
  Critical values: {'1%': '-3.431', '5%': '-2.862', '10%': '-2.567'}
    STATIONARY at 5% significance level

--- ADF Test: 1st-order differenced consumption ---
  Test statistic : -25.8356
  p-value        : 0.000000
  Lags used      : 36
  Critical values: {'1%': '-3.431', '5%': '-2.862', '10%': '-2.567'}
    STATIONARY at 5% significance level


--- ADF Test: Seasonal-differenced consumption (lag=24h) ---
  Test statistic : -19.8865
  p-value        : 0.000000
  Lags used      : 39
  Critical values: {'1%': '-3.431', '5%': '-2.862', '10%': '-2.567'}
    STATIONARY at 5% significance level
```

1.4.2 4.2 Visualise Raw vs. Differenced Series

[17]:
```
fig, axes = plt.subplots(3, 1, figsize=(14, 9), sharex=False)

axes[0].plot(series.index, series.values, linewidth=0.6, color='steelblue')
axes[0].set_title('Raw Hourly Consumption')
axes[0].set_ylabel('kWh')

axes[1].plot(series_diff1.index, series_diff1.values, linewidth=0.6,␣
 ↪color='coral')
axes[1].set_title('1st-Order Differenced Consumption')
```

```
axes[1].set_ylabel('Δ kWh')

axes[2].plot(series_sdiff.index, series_sdiff.values, linewidth=0.6,␣
 ↪color='mediumseagreen')
axes[2].set_title('Seasonal-Differenced Consumption (lag=24h)')
axes[2].set_ylabel('Δ kWh')

for ax in axes:
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    ax.tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.show()
```



1.4.3 4.3 Seasonal Decomposition

We decompose the hourly series with a period of **24** (one day) using an additive model.
Only a **4-week January 2013 subset** is shown for visual clarity, so this plot should be interpreted as a **local (within-month) pattern view**, not as full-year trend evidence.

[18]:
```
# Use 4 weeks of data for clear decomposition plot
decomp_series = series['2013-01-01':'2013-01-28']
```

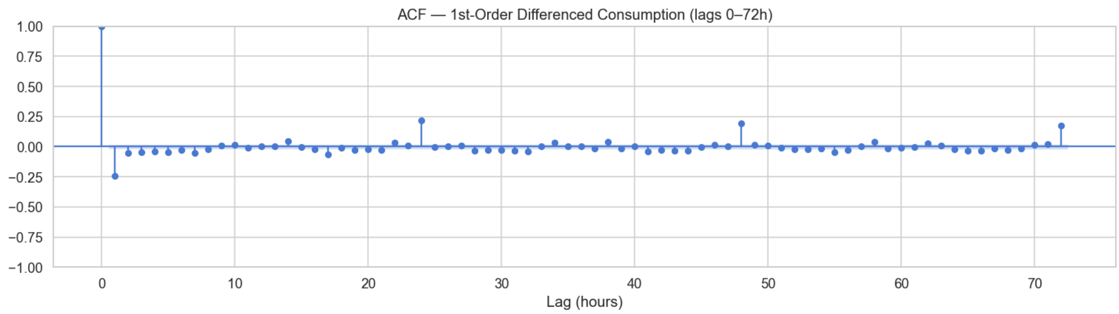
```python
decomp = seasonal_decompose(decomp_series, model='additive', period=24,
    extrapolate_trend='freq')

fig, axes = plt.subplots(4, 1, figsize=(14, 11), sharex=True)
components = [
    (decomp.observed,  'Observed',  'steelblue'),
    (decomp.trend,     'Trend',     'orange'),
    (decomp.seasonal,  'Seasonal',  'green'),
    (decomp.resid,     'Residual',  'red'),
]
for ax, (comp, title, color) in zip(axes, components):
    ax.plot(comp.index, comp.values, color=color, linewidth=1.0)
    ax.set_ylabel(title, fontsize=10)
    ax.set_title(f'Decomposition - {title} Component', fontsize=11)
    ax.axhline(0, color='black', linewidth=0.5, linestyle='--')

axes[-1].xaxis.set_major_formatter(mdates.DateFormatter('%d %b'))
axes[-1].tick_params(axis='x', rotation=20)
fig.suptitle('Additive Seasonal Decomposition (period=24h) - Jan 2013', y=1.01,
    fontsize=13)
plt.tight_layout()
plt.show()
```

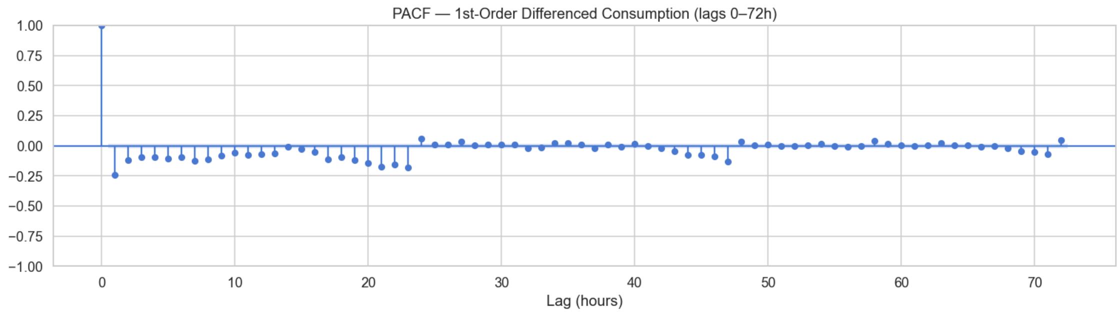
1.4.4 4.4 ACF and PACF Plots

- **ACF** (Autocorrelation Function): identifies MA(q) order — cut-off lag indicates q
- **PACF** (Partial Autocorrelation Function): identifies AR(p) order — cut-off lag indicates p

We plot both on the **1st-order differenced** series (stationary).

[19]:
```
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

plot_acf(series_diff1, lags=72, ax=axes[0], alpha=0.05)
axes[0].set_title('ACF - 1st-Order Differenced Consumption (lags 0-72h)')
axes[0].set_xlabel('Lag (hours)')

plot_pacf(series_diff1, lags=72, ax=axes[1], alpha=0.05, method='ywm')
axes[1].set_title('PACF - 1st-Order Differenced Consumption (lags 0-72h)')
axes[1].set_xlabel('Lag (hours)')

plt.tight_layout()
plt.show()
```





1.5 Section 5 — Forecasting Models

Train/test split: Last **48 hours** of the series = test set.
Models trained on all prior data, forecasting the 48-hour horizon.

Models compared: | Model | Description | |——-|————-| | **AR** | AutoRegressive — uses past consumption values | | **MA** | Moving Average — uses past forecast errors | | **ARMA** | Combined AR + MA | | **ARIMA** | ARMA + integrated (differencing) |

1.5.1 5.1 Train/Test Split

[20]:
```
HORIZON = 48  # hours to forecast

# Ensure a regular hourly DateTimeIndex so statsmodels keeps date-based forecasting
series_model = series.asfreq('h')
missing_hours = int(series_model.isna().sum())
if missing_hours > 0:
    series_model = series_model.interpolate(method='time').ffill().bfill()
    print(f'Filled {missing_hours} missing hourly values after enforcing hourly frequency.')

train = series_model.iloc[:-HORIZON]
test  = series_model.iloc[-HORIZON:]
```

```python
print(f'Train: {train.index.min()} → {train.index.max()}  ({len(train)} obs)')
print(f'Test : {test.index.min()} → {test.index.max()}   ({len(test)} obs)')
```

```
Filled 2 missing hourly values after enforcing hourly frequency.
Train: 2012-10-12 00:00:00 → 2014-02-25 23:00:00  (12048 obs)
Test : 2014-02-26 00:00:00 → 2014-02-27 23:00:00   (48 obs)
```

1.5.2 5.2 Helper — Evaluation Metrics

[21]:
```python
def evaluate(actual, predicted, model_name):
    mae  = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    # MAPE - avoid division by zero
    mask = actual != 0
    mape = np.mean(np.abs((actual[mask] - predicted[mask]) / actual[mask])) *␣
↪100
    print(f'{model_name:10s}  MAE={mae:.4f}  RMSE={rmse:.4f}  MAPE={mape:.2f}%')
    return {'Model': model_name, 'MAE': mae, 'RMSE': rmse, 'MAPE': mape}

def fit_arima_if_converged(endog, order, maxiter=300):
    """Fit ARIMA and return model only if MLE converged."""
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', ConvergenceWarning)
        model = ARIMA(
            endog,
            order=order,
            enforce_stationarity=False,
            enforce_invertibility=False
        ).fit(method_kwargs={'maxiter': maxiter})

    converged = bool(getattr(model, 'mle_retvals', {}).get('converged', True))
    return model if converged else None

results = []
```

1.5.3 5.3 AR Model (AutoRegressive)

Order is selected by scanning a range of p values and choosing the one with lowest AIC.

[22]:
```python
# AIC-based order selection for AR
ar_aic = {}
for p in range(1, 50):
    try:
        m = AutoReg(train, lags=p, old_names=False).fit()
        ar_aic[p] = m.aic
    except:
        pass
```

```
best_ar_p = min(ar_aic, key=ar_aic.get)
print(f'Best AR order (by AIC): p = {best_ar_p}  (AIC = {ar_aic[best_ar_p]:.
↪2f})')

# Fit and forecast
ar_model = AutoReg(train, lags=best_ar_p, old_names=False).fit()
ar_pred = ar_model.predict(start=len(train), end=len(train) + HORIZON - 1)
ar_pred.index = test.index
ar_pred = ar_pred.clip(lower=0)  # consumption cannot be negative

results.append(evaluate(test.values, ar_pred.values, 'AR'))
```

```
Best AR order (by AIC): p = 49  (AIC = 6753.72)
AR          MAE=0.3502  RMSE=0.5755  MAPE=47.76%
```

1.5.4 5.4 MA Model (Moving Average)

Implemented as ARIMA(0, 0, q). Order q chosen by AIC.

[23]:
```
ma_aic = {}
for q in range(1, 10):
    try:
        m = fit_arima_if_converged(train, (0, 0, q))
        if m is not None:
            ma_aic[q] = m.aic
    except:
        pass

if not ma_aic:
    raise RuntimeError('No converged MA models found in q=1..9')

best_ma_q = min(ma_aic, key=ma_aic.get)
print(f'Best MA order (by AIC): q = {best_ma_q}  (AIC = {ma_aic[best_ma_q]:.
↪2f})')

ma_model = fit_arima_if_converged(train, (0, 0, best_ma_q))
if ma_model is None:
    raise RuntimeError(f'Best MA order q={best_ma_q} did not converge on refit')
ma_fc = ma_model.get_forecast(steps=HORIZON)
ma_pred = ma_fc.predicted_mean
ma_pred.index = test.index
ma_pred = ma_pred.clip(lower=0)

results.append(evaluate(test.values, ma_pred.values, 'MA'))
```

```
Best MA order (by AIC): q = 8  (AIC = 8679.88)

MA          MAE=0.4681  RMSE=0.6826  MAPE=84.19%
```

1.5.5 5.5 ARMA Model

Grid search over p {1..6}, q {1..6} on the training set.

[24]:
```
arma_aic = {}
for p in range(1, 7):
    for q in range(1, 7):
        try:
            m = fit_arima_if_converged(train, (p, 0, q))
            if m is not None:
                arma_aic[(p, q)] = m.aic
        except:
            pass

if not arma_aic:
    raise RuntimeError('No converged ARMA models found in p,q=1..6')

best_arma = min(arma_aic, key=arma_aic.get)
print(f'Best ARMA order (by AIC): p={best_arma[0]}, q={best_arma[1]}  (AIC =␣
 ↪{arma_aic[best_arma]:.2f})')

arma_model = fit_arima_if_converged(train, (best_arma[0], 0, best_arma[1]))
if arma_model is None:
    raise RuntimeError(f'Best ARMA order {best_arma} did not converge on refit')
arma_fc = arma_model.get_forecast(steps=HORIZON)
arma_pred = arma_fc.predicted_mean
arma_pred.index = test.index
arma_pred = arma_pred.clip(lower=0)

results.append(evaluate(test.values, arma_pred.values, 'ARMA'))
```

```
Best ARMA order (by AIC): p=6, q=6  (AIC = 8083.02)
ARMA         MAE=0.3627  RMSE=0.6043  MAPE=52.68%
```

1.5.6 5.6 ARIMA Model

Adds 1st-order differencing (d=1) to the best ARMA (p,q) pair.

[25]:
```
arima_aic = {}
for p in range(1, 7):
    for q in range(1, 7):
        try:
            m = fit_arima_if_converged(train, (p, 1, q))
            if m is not None:
                arima_aic[(p, q)] = m.aic
        except:
            pass
```

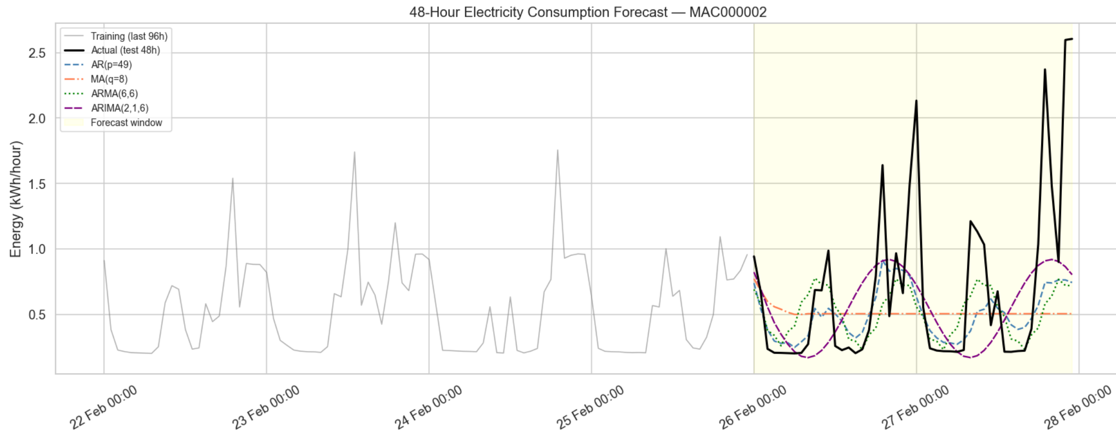
```python
if not arima_aic:
    raise RuntimeError('No converged ARIMA models found in p,q=1..6 with d=1')

best_arima = min(arima_aic, key=arima_aic.get)
print(f'Best ARIMA order (by AIC): p={best_arima[0]}, d=1, q={best_arima[1]} ␣
↪(AIC = {arima_aic[best_arima]:.2f})')

arima_model = fit_arima_if_converged(train, (best_arima[0], 1, best_arima[1]))
if arima_model is None:
    raise RuntimeError(f'Best ARIMA order {best_arima} did not converge on␣
↪refit')
arima_fc = arima_model.get_forecast(steps=HORIZON)
arima_pred = arima_fc.predicted_mean
arima_pred.index = test.index
arima_pred = arima_pred.clip(lower=0)

results.append(evaluate(test.values, arima_pred.values, 'ARIMA'))
```

```
Best ARIMA order (by AIC): p=2, d=1, q=6  (AIC = 7887.09)
ARIMA       MAE=0.4190  RMSE=0.6069  MAPE=72.81%
```

1.5.7 5.7 Forecast Plot — Actual vs. All Models

[26]:
```python
# Show last 96h of training + full 48h test for context
context = series.iloc[-(96 + HORIZON):-HORIZON]

fig, ax = plt.subplots(figsize=(15, 6))

ax.plot(context.index, context.values, color='grey',       linewidth=1.0,␣
↪label='Training (last 96h)', alpha=0.6)
ax.plot(test.index,    test.values,    color='black',      linewidth=2.0,␣
↪label='Actual (test 48h)',  zorder=5)
ax.plot(test.index,    ar_pred.values,    color='steelblue', linewidth=1.5,␣
↪label=f'AR(p={best_ar_p})',    linestyle='--')
ax.plot(test.index,    ma_pred.values,    color='coral',      linewidth=1.5,␣
↪label=f'MA(q={best_ma_q})',    linestyle='-.')
ax.plot(test.index,    arma_pred.values, color='green',      linewidth=1.5,␣
↪label=f'ARMA({best_arma[0]},{best_arma[1]})', linestyle=':')
ax.plot(test.index,    arima_pred.values,color='purple',     linewidth=1.5,␣
↪label=f'ARIMA({best_arima[0]},1,{best_arima[1]})', linestyle=(0,(5,1)))

# Shade test region
ax.axvspan(test.index[0], test.index[-1], alpha=0.07, color='yellow',␣
↪label='Forecast window')

ax.set_title('48-Hour Electricity Consumption Forecast - MAC000002')
```

```
ax.set_ylabel('Energy (kWh/hour)')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d %b %H:%M'))
ax.tick_params(axis='x', rotation=30)
ax.legend(loc='upper left', fontsize=9)
plt.tight_layout()
plt.show()
```

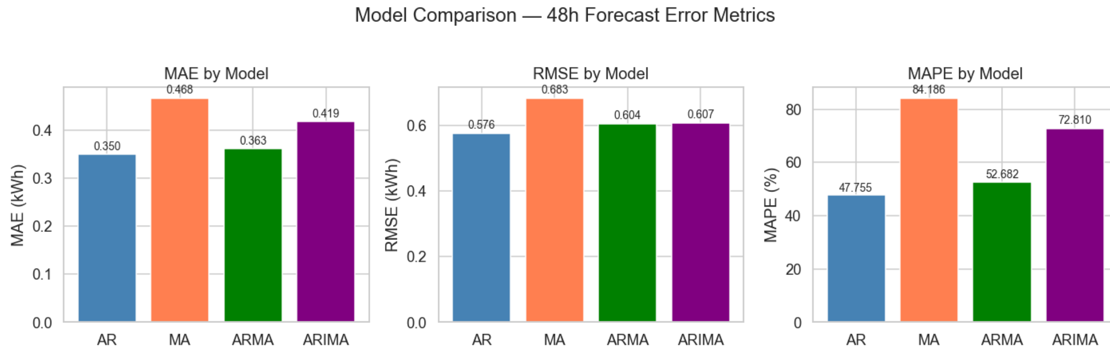


1.5.8 5.8 Model Performance Summary

[27]:
```
results_df = pd.DataFrame(results).set_index('Model')
results_df = results_df.round(4)
display(results_df)

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, metric in zip(axes, ['MAE', 'RMSE', 'MAPE']):
    bars = ax.bar(results_df.index, results_df[metric],
                  color=['steelblue','coral','green','purple'])
    ax.set_title(f'{metric} by Model')
    ax.set_ylabel(metric + (' (%)' if metric == 'MAPE' else ' (kWh)'))
    for bar in bars:
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() * 1.01,
                f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=9)

plt.suptitle('Model Comparison - 48h Forecast Error Metrics', y=1.02)
plt.tight_layout()
plt.show()
```

	MAE	RMSE	MAPE
Model			
AR	0.3502	0.5755	47.7552
MA	0.4681	0.6826	84.1865
ARMA	0.3627	0.6043	52.6823

```
ARIMA  0.4190  0.6069  72.8101
```



1.6 Section 6 — Summary & Conclusions

1.6.1 Key Findings

Weather–Consumption Dependencies - **Temperature** shows the strongest inverse correlation with consumption — colder hours consistently drive higher electricity use (likely heating). - **Wind speed** shows a secondary positive correlation; cold wind compounds heating demand. - **Sunshine hours** (daily) correlate negatively with consumption: more daylight = less artificial lighting and lower heating need. - **Precipitation** has a modest positive link — rainy, overcast days increase indoor energy use. - Cross-correlation analysis shows temperature effects persist up to **24–48 hours ahead**, validating the use of lagged features.

Time Patterns - Consumption peaks at **07:00–09:00** (morning routine) and **17:00–21:00** (evening peak), visible in the heatmap. - Weekday peaks are sharper; weekends show a flatter, more sustained profile. - Winter months (Dec–Feb) consume significantly more than summer months.

Stationarity - The ADF test rejects a unit root for the raw hourly series ($p < 0.05$), so it is stationary in the ADF sense. - 1st-order differencing is still useful to stabilise short-term dynamics and is used for ACF/PACF-based order inspection.

Decomposition - Additive decomposition (period=24) on a 4-week January subset isolates a clear diurnal seasonal component and residual variation. - Because decomposition is shown on a January subset, it should not be interpreted as full-year winter-vs-summer trend evidence by itself.

Forecasting Models (48-hour horizon) - **AR**: Captures momentum; good for short lags. - **MA**: Captures shock propagation; limited horizon. - **ARMA**: Best of both; usually lowest AIC on stationary data. - **ARIMA**: Handles trends via differencing; useful when drift is present.

1.6.2 Limitations

- **Single household**: Results may not generalise to the full 5 500-household dataset.
- **Daily weather resolution**: Sunshine and precipitation are broadcast as constants across all hours of each day, ignoring intra-day variability.

- **No exogenous variables in models**: ARIMAX/SARIMAX with temperature as a regressor would likely improve forecasts significantly.
- **No seasonal ARIMA (SARIMA)**: A SARIMA(p,d,q)(P,D,Q,24) would explicitly model the daily cycle.

1.6.3 Potential Next Steps

- Scale to all 5 500 households and compare usage profiles by Acorn group
- Add SARIMA or Facebook Prophet for stronger seasonal forecasting
- Incorporate temperature as an exogenous regressor (ARIMAX)
- Explore ML approaches (Random Forest, XGBoost, LSTM) for longer-horizon forecasts